KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Personal Expense Tracking and Monthly Budget Management System

Submitted by

CHRISTY KIRUBA J

25BCS014

Under the Guidance of

Dr . BOOPALAN SUNDARAMOORTHY

Technical head

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Personal Expense Tracking and Monthly Budget Management System**" submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of **Dr. BOOPALAN SUNDARAMOORTHY**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

CHRISTY KIRUBA.J

CERTIFICATE

This is to certify that the project entitled **"Personal Expense Tracking and Monthly Budget Management System"** is a **bona fide work** carried out by **CHRISTY KIRUBA J I – BSC CS 'A'**, **Department of computer science**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **JAVA PROGRAMMING**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place:

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical head, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	**6**
1.1	Objective	6
1.2	Overview	6
1.3	Features	6
2	**Program Analysis**	8
2.1	Problem Definition	8
2.2	Project Overview	8
2.3	Scopes and Limitations	8
3	**System Design**	**10**
3.1	Modules	10
4	**Methodology**	**12**
4.1	Problem Analysis	12
4.2	System Design	12
4.3	Program Development	13
4.4	Testing and Validation	13
4.5	Result and Documentation	13
4.6	Algorithm	13
4.7	Flowchart	14
5	**Source Code**	**15**
6	**Input and Output Screens**	**22**
7	**Discussion and Conclusion**	**24**
7.1	Discussion	24
7.2	Conclusion	24
8	**References**	**25**

ABSTRACT

In an era of increasing digital transactions and fluctuating economic conditions, maintaining financial discipline has become a significant challenge for individuals. This project introduces a Personal Expense Tracking and Monthly Budget Management System, a comprehensive digital solution designed to help users gain full transparency over their financial health.

Core Objectives

The primary goal of this system is to bridge the gap between earning and intentional spending. It moves beyond simple record-keeping to provide actionable insights into spending behavior. Key objectives include:

Real-time Tracking: Categorizing expenses (e.g., housing, food, entertainment) as they occur.

Budget Adherence: Setting monthly limits for specific categories and providing automated alerts when thresholds are approached.

Data Visualization: Utilizing interactive charts to illustrate the ratio of savings to expenditures.

Technical Implementation

The system is built on a robust architecture to ensure data integrity and user privacy.

Frontend: A user-centric interface focused on ease of data entry and clarity.

Backend: Logic for calculating recurring expenses, identifying spending trends, and managing user authentication.

Database: Structured storage (SQL/NoSQL) to maintain historical financial records for long-term trend analysis.

1. INTRODUCTION

In today’s fast-paced economic environment, maintaining financial health requires more than just earning an income; it requires disciplined oversight of where every unit of currency is allocated. A Personal Expense Tracking and Monthly Budget Management System is a digital solution designed to help individuals transition from passive spending to active financial planning. By centralizing all transactions into a single interface, the system provides a clear picture of one's financial standing, helping to eliminate "leaky" spending habits and foster long-term wealth accumulation.

2. Objective

The primary goal of this system is to empower users with financial literacy and control. Specific objectives include:

Financial Visibility: To provide a real-time, comprehensive view of income versus expenditures.

Goal Alignment: To ensure that monthly spending aligns with long-term financial goals, such as savings, investments, or debt repayment.

Spending Accountability: To categorize and monitor discretionary spending, making it easier to identify and reduce unnecessary costs.

Data-Driven Decisions: To generate historical reports that allow users to predict future expenses and adjust their lifestyle accordingly.

3. Overview

The system acts as a personal financial hub where users can input their monthly income and log daily expenses across various categories. It operates on a Zero-Based Budgeting principle, where every dollar is assigned a purpose. The architecture typically consists of a user-friendly dashboard that visualizes the remaining budget for the month. By comparing "Budgeted" amounts against "Actual" spending, the system alerts the user to potential overages before they occur, ensuring that the user never spends more than they earn.

4. Features

Income & Expense Logging: Simple, intuitive forms to record transactions with dates, amounts, and descriptions.

Custom Categorization: Ability to group expenses into specific buckets such as Housing, Transportation, Groceries, Utilities, and Entertainment.

Automated Budget Alerts: Real-time notifications when a user reaches 80% or 100% of a specific category’s monthly limit.

Visual Analytics: Interactive charts (pie charts for spending distribution and bar graphs for monthly trends) to simplify complex financial data.

Savings Goal Tracker: A dedicated module to set targets for specific milestones (e.g., emergency funds or vacation savings) and track progress over time.

Secure Data Export: Capability to generate and export monthly PDF or CSV reports for deeper analysis or tax purposes.

2. PROGRAM ANALYSIS

1. Problem Definition

Many individuals struggle with financial instability not due to a lack of income, but due to a lack of financial awareness. Without a structured system, several critical issues arise:

Manual Tracking Inefficiency: Relying on memory or physical receipts often leads to forgotten expenses and inaccurate balances.

Lack of Categorization: Difficulty in identifying which areas (e.g., dining out vs. utilities) are consuming the largest portion of income.

Overspending: Without real-time updates, users often exceed their financial limits before the month ends, leading to debt or reliance on credit.

Poor Savings Discipline: Savings are often treated as an afterthought—whatever is "left over"—rather than a planned priority.

2. Project Overview

This project involves the development of a centralized digital platform that automates the monitoring of cash flow. The system serves as a bridge between earning and saving by providing a structured framework for budget allocation. It is designed to be a lightweight yet robust application where users can define their monthly financial boundaries. By integrating automated calculations and visual feedback, the project simplifies complex accounting into a daily tool that encourages consistent financial habits and long-term fiscal responsibility.

3. Scopes and Limitations

Scope (What the system does):

User Profile Management: Secure login and personalized dashboard for individual users.

Dynamic Budgeting: Allows users to set specific spending limits for different categories each month.

Transaction History: Maintains a searchable and filterable database of all past income and expenditures.

Graphical Reporting: Generates visual summaries (e.g., pie charts and trend lines) to illustrate spending patterns.

Recursive Expense Management: Ability to set up recurring entries for fixed costs like rent or subscriptions.

Limitations (What the system does not do):

No Real-Time Bank Integration: The system requires users to manually input data (unless integrated with specific APIs, which may be outside the current project phase).

No Investment Trading: The system tracks expenses but does not function as a platform for buying or selling stocks and crypto.

Single-User Focus: In its standard version, it is designed for individual use rather than managing complex multi-user corporate accounts.

Internet Dependency: If built as a web or cloud-based app, the system may require an active connection to sync data across devices.

3. SYSTEM DESIGN

1. System Architecture

The application is structured into three distinct layers:

Presentation Layer (Frontend): The interface where users interact with the system. Built using HTML/CSS/JavaScript or Java Swing/JavaFX, it focuses on ease of data entry and clear visualization.

Application Layer (Backend): The core logic of the system. It handles the processing of inputs, calculation of fines or balances, and enforces budgeting rules. (e.g., Java or Python).

Data Layer (Database): A structured storage system (such as MySQL or SQLite) that holds user credentials, transaction history, and budget configurations.

2. Functional Modules

The system is divided into logical components to ensure scalability:

Authentication Module: Handles user registration, login, and session management to keep financial data private.

Transaction Engine: The "calculator" of the system. It adds income, subtracts expenses, and updates the "Total Balance" in real-time.

Budgeting Logic: Compares the sum of transactions in a specific category against the pre-set limit and triggers alerts if the limit is breached.

Reporting Engine: Queries the database to aggregate data by week or month, converting raw numbers into visual charts.

3. System Flow (Process Diagram)

Input: The user enters a transaction (e.g., $50 for "Groceries").

Validation: The system checks if the category exists and if the amount is a valid positive number.

Processing:

The transaction is saved to the Database.

The Total Balance is recalculated.

The Category Budget is updated (e.g., Groceries budget reduced by $50).

Output: The dashboard refreshes to show the new balance and updated progress bars for spending.

4. Design Constraints & Security

Data Persistence: All data must be saved locally or in the cloud so that information is not lost when the application is closed.

Security: Implementation of basic encryption for passwords and local data files to prevent unauthorized access to financial history.

Input Sanitization: Ensuring that users cannot enter text into numerical fields, preventing system crashes.

4. METHODOLOGY

The development process follows a structured software development life cycle to ensure the system is reliable and user-friendly. The methodology includes:

Problem Analysis: Identifying current financial tracking challenges.

System Design: Planning the modules and logical structure.

Program Development: Implementing the logic using Java.

Testing and Validation: Ensuring the math and inputs are accurate.

Result and Documentation: Preparing the final report and user guide.

4.1. PROBLEM ANALYSIS

Managing finances manually often leads to several difficulties that this system aims to solve:

Manual Entry Errors: Handwritten logs often contain calculation mistakes.

Lack of Real-Time Data: Difficult to know exactly how much budget remains mid-month.

Categorization Difficulty: Hard to track which specific areas (food, bills, fun) consume the most money.

Inefficiency: Sorting through receipts and notes is time-consuming.

4.2. SYSTEM DESIGN

The system is divided into functional modules to handle different parts of budget management:

Budget Initialization Module: Sets the starting monthly limit.

Expense Input Module: Captures the amount and category of each spending event.

Calculation Engine: Performs the logic of Remaining Budget = Total Budget - Total Expenses.

Reporting Module: Generates the final summary showing savings and spending percentages.

4.3. PROGRAM DEVELOPMENT

The application is built using Core Java, focusing on clean logic and user interaction:

Classes and Objects: Representing "Expenses" and "Budgets" as distinct entities.

Conditional Statements: Checking if an expense exceeds the remaining budget.

Loops: Allowing the user to enter multiple expenses in one session.

Scanner Class: Used for capturing real-time user inputs for amounts and categories.

4.4. RESULT AND DOCUMENTATION

Upon completion, the system provides a clear financial overview for the month. Documentation includes screenshots of the menu selection, expense entries, and the final "Monthly Budget Status" report. This serves as proof that the application automates financial tracking effectively.

4.5. ALGORITHM

Start the program.

Display the main menu.

Input the total monthly budget amount.

Enter a new expense amount and choose a category.

Calculate the new balance: Budget = Budget - Expense.

Store the entry in the transaction log.

Ask if the user wants to add more items; if yes, return to Step 4.

Display the final summary of all categories and remaining funds.

End the program.

4.6. FLOWCHART

The flow follows a linear path from initialization to reporting:

Start → Set Monthly Budget → Add Expense? → (Yes) → Subtract Amount → Update Log → (No) → Generate Report → End.

5.SOURCE CODE

```
import java.util.*;


// Transaction class
class Transaction {
    String date;
    String type; // Income or Expense
    String category;
    String description;
    double amount;


    public Transaction(String date, String type, String category, String description, double amount) {
        this.date = date;
        this.type = type;
        this.category = category;
        this.description = description;
        this.amount = amount;
    }


    public void display() {
        System.out.println(date + " | " + type + " | " + category + " | " + description + " | ₹" + amount);
    }
}
```

```
// Main class
public class ExpenseManager {

    static ArrayList<Transaction> transactions = new ArrayList<>();
    static double balance = 0;
    static double monthlyBudget = 0;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n====== PERSONAL EXPENSE TRACKER ======");
            System.out.println("1. Set Monthly Budget");
            System.out.println("2. Add Income");
            System.out.println("3. Add Expense");
            System.out.println("4. View All Transactions");
            System.out.println("5. View Balance");
            System.out.println("6. Monthly Summary");
            System.out.println("7. Exit");
            System.out.print("Enter choice: ");

            choice = sc.nextInt();
            sc.nextLine(); // clear buffer

            switch (choice) {
```

```
case 1:
    System.out.print("Enter Monthly Budget: ");
    monthlyBudget = sc.nextDouble();
    System.out.println("Budget set successfully!");
    break;

case 2:
    addIncome(sc);
    break;

case 3:
    addExpense(sc);
    break;

case 4:
    viewTransactions();
    break;

case 5:
    System.out.println("Current Balance: ₹" + balance);
    break;

case 6:
    monthlySummary();
    break;

case 7:
```

```
                System.out.println("Thank you for using the system!");
                break;

            default:
                System.out.println("Invalid choice!");
        }

    } while (choice != 7);

    sc.close();
}

// Add Income
public static void addIncome(Scanner sc) {
    System.out.print("Enter Date (dd-mm-yyyy): ");
    String date = sc.nextLine();

    System.out.print("Enter Source: ");
    String source = sc.nextLine();

    System.out.print("Enter Amount: ");
    double amount = sc.nextDouble();
    sc.nextLine();

    transactions.add(new Transaction(date, "Income", source, "Income Added", amount));
    balance += amount;
```

```
    System.out.println("Income added successfully!");
}

// Add Expense
public static void addExpense(Scanner sc) {
    System.out.print("Enter Date (dd-mm-yyyy): ");
    String date = sc.nextLine();

    System.out.print("Enter Category (Food/Travel/etc): ");
    String category = sc.nextLine();

    System.out.print("Enter Description: ");
    String desc = sc.nextLine();

    System.out.print("Enter Amount: ");
    double amount = sc.nextDouble();
    sc.nextLine();

    transactions.add(new Transaction(date, "Expense", category, desc, amount));
    balance -= amount;

    System.out.println("Expense added successfully!");
}

// View Transactions
public static void viewTransactions() {
    System.out.println("\n--- Transaction History ---");
```

```
    if (transactions.isEmpty()) {
        System.out.println("No transactions found!");
    } else {
        for (Transaction t : transactions) {
            t.display();
        }
    }
}

// Monthly Summary
public static void monthlySummary() {
    double totalIncome = 0;
    double totalExpense = 0;

    for (Transaction t : transactions) {
        if (t.type.equals("Income")) {
            totalIncome += t.amount;
        } else {
            totalExpense += t.amount;
        }
    }

    System.out.println("\n====== MONTHLY SUMMARY ======");
    System.out.println("Total Income  : ₹" + totalIncome);
    System.out.println("Total Expense : ₹" + totalExpense);
    System.out.println("Balance       : ₹" + balance);
```

```
        System.out.println("Budget        : ₹" + monthlyBudget);

        if (monthlyBudget > 0) {
            if (totalExpense > monthlyBudget) {
                System.out.println(" You exceeded your budget!");
            } else {
                System.out.println("You are within budget!");
            }
        }
    }
}
```

\

6.INPUT AND OUTPUT SCREENS

```
====== PERSONAL EXPENSE TRACKER ======
1. Set Monthly Budget
2. Add Income
3. Add Expense
4. View All Transactions
5. View Balance
6. Monthly Summary
7. Exit
Enter choice: 1
Enter Monthly Budget: 35000
Budget set successfully!

====== PERSONAL EXPENSE TRACKER ======
1. Set Monthly Budget
2. Add Income
3. Add Expense
4. View All Transactions
5. View Balance
6. Monthly Summary
7. Exit
Enter choice: 2
Enter Date (dd-mm-yyyy): 14/11/2007
Enter Source: bank
Enter Amount: 2500
Income added successfully!

====== PERSONAL EXPENSE TRACKER ======
1. Set Monthly Budget
2. Add Income
3. Add Expense
4. View All Transactions
5. View Balance
6. Monthly Summary
7. Exit
Enter choice: 3
Enter Date (dd-mm-yyyy): 17/11/2007
Enter Category (Food/Travel/etc): food
Enter Description: 400
Enter Amount: 500
Expense added successfully!
```

```
====== PERSONAL EXPENSE TRACKER ======
1. Set Monthly Budget
2. Add Income
3. Add Expense
4. View All Transactions
5. View Balance
6. Monthly Summary
7. Exit
Enter choice: 4

--- Transaction History ---
14/11/2007 | Income | bank | Income Added | ?2500.0
17/11/2007 | Expense | food | 400 | ?500.0

====== PERSONAL EXPENSE TRACKER ======
1. Set Monthly Budget
2. Add Income
3. Add Expense
4. View All Transactions
5. View Balance
6. Monthly Summary
7. Exit
Enter choice: 5
Current Balance: ?2000.0
```

```
====== PERSONAL EXPENSE TRACKER ======
1. Set Monthly Budget
2. Add Income
3. Add Expense
4. View All Transactions
5. View Balance
6. Monthly Summary
7. Exit
Enter choice: 6

====== MONTHLY SUMMARY ======
Total Income  : ?2500.0
Total Expense : ?500.0
Balance       : ?2000.0
Budget        : ?35000.0
 You are within budget!

====== PERSONAL EXPENSE TRACKER ======
1. Set Monthly Budget
2. Add Income
3. Add Expense
4. View All Transactions
5. View Balance
6. Monthly Summary
7. Exit
Enter choice: 7
Thank you for using the system!

D:\JAVA FOLDER>
```

7.DISCUSSION & CONCLUSION

7.1 Discussion

The Personal Expense Tracking and Monthly Budget Management System was developed to automate the monitoring of individual finances and monthly savings. The system allows users to input a total monthly budget, log various expense categories, and generate financial summaries automatically. During the development process, Core Java programming concepts such as loops, conditional statements, methods, and object-oriented principles were applied.

The system successfully performs balance calculations, subtracting daily expenditures from the initial budget to show real-time financial status. The application was tested with various inputs to ensure that it handles numerical data accurately and manages budget overflows correctly. The results indicate that using this automated tool significantly reduces manual effort and improves the accuracy of personal financial record-keeping compared to traditional methods.

7.2 Conclusion

The Personal Expense Tracking and Monthly Budget Management System successfully demonstrates the practical application of Core Java programming concepts in developing a real-world financial utility. The system effectively automates budget tracking, reduces the risk of calculation errors, and provides users with a clear view of their spending habits. It simplifies monthly financial management by generating structured summaries based on user entries.

This project assists in understanding how Java programming can be utilized to create software that solves common personal organization challenges. The project can be further enhanced in the future by adding features such as persistent database connectivity to save records across multiple months, a graphical user interface (GUI) using Java Swing, or integrated data visualization tools like charts and graphs.

8.REFERENCES

The following resources were used during the development of this project:

Books

Herbert Schildt – Java: The Complete Reference

Websites

Oracle Java Documentation: https://www.oracle.com/java

GeeksforGeeks: https://www.geeksforgeeks.org

W3Schools: https://www.w3schools.com

JavaTpoint: https://www.javatpoint.com